



TEMA 092. DISEÑO ARQUITECTÓNICO DE SISTEMAS. DIAGRAMAS DE DESPLIEGUE.

Actualizado a 09/10/2020

1. DISEÑO DE LA ARQUITECTURA DEL SOFTWARE

La arquitectura de un sistema software constituye un modelo relativamente pequeño e intelectualmente comprensible de cómo está estructurado dicho sistema y de cómo trabajan juntos sus componentes. Es la representación de dicho software que permite:

- Analizar la efectividad del diseño para la consecución de los requisitos fijados.
- Considerar las alternativas arquitectónicas en una etapa en la cual hacer cambios en el diseño es relativamente fácil.
- Reducir los riesgos asociados a la construcción del software

1.1. DISEÑO ARQUITECTÓNICO Y ESTILOS ARQUITECTÓNICOS

El diseño arquitectónico se centra en la representación de la estructura de los componentes del software, sus propiedades e interacciones. Para ello puede seguir diferentes estilos arquitectónicos, que son conjuntos de principios que definen a alto nivel un aspecto de la aplicación, siendo los principales aspectos: comunicación, despliegue, dominio, interacción, relación y estructura.

Cada estilo arquitectónico describe una categoría del sistema que contiene: un conjunto de componentes que realizan una función requerida por el sistema; un conjunto de conectores que posibilitan la comunicación, la coordinación y la cooperación entre los componentes; restricciones que definen como se puede integrar los componentes que forman el sistema; y modelos semánticos que permiten al diseñador entender las propiedades globales de un sistema para analizar las propiedades conocidas de sus partes constituyentes.

La clasificación de estilos arquitectónicos es muy diversa, por lo que suele agruparse en grandes grupos teniendo en cuenta diversas características del sistema, englobando cada uno de estos grupos una serie de estilos arquitectónicos que comparten características. Generalmente los estilos proveen guías para crear una clase amplia de arquitectura, donde los patrones se enmarcan en darle solución a problemas más pequeños y más específicos dentro de un estilo dado.

1.2. CLASIFICACIÓN DE ESTILOS ARQUITECTÓNICOS

ARQUITECTURAS CENTRADAS EN LOS DATOS

En el centro de estas arquitecturas se encuentra un almacén de datos al que otros componentes acceden con frecuencia para actualizar, añadir, borrar o modificar sus datos. El software del cliente accede a dicho almacén central fundamentalmente de dos posibles modos:

MODELO DE REPOSITORIO (SHARED-DATA)

Es un modelo en el que existe un almacenamiento central de datos (BD) y un conjunto de componentes que operan sobre éste. La entrada de los datos es realizada por los componentes a lo cual la BD responde de cierta manera. Este modelo permite compartir grandes cantidades de datos con un vasto número de componentes, los cuales desconocen el uso que hagan de dichos datos otros componentes. Sin embargo, a medida que crece el número de componentes, el acceso, recuperación y uso pueden entrar en conflicto ya que todos deben manejar las mismas políticas para realizar dichas actividades. De igual manera, al tener un repositorio global, los cambios en la estructura de los datos pueden afectar a todos los componentes.

MODELO DE PIZARRA (BLACKBOARD)

En este modelo, a diferencia del modelo de repositorio, el almacén de datos es activo y los componentes son pasivos. Ante cualquier cambio en los datos, el almacén notifica a los componentes. Implementando componentes computacionales que hacen uso de AI, el almacén de datos se autorregula sin necesidad de eventos externos.

ARQUITECTURAS DE FLUJO DE DATOS

Se aplica cuando los datos de entrada son transformados a través de una serie de componentes computacionales o manipulativos en los datos de salida. Ejemplos de este estilo son las arquitecturas de tuberías y filtros y las de proceso secuencial por lotes (batch).

TUBERÍAS Y FILTROS

En esta arquitectura, las tuberías corresponden a los conectores y los filtros a los componentes. Tiene como objetivo el aseguramiento de las cualidades de reutilización y modificabilidad. Este estilo se caracteriza por ver el sistema como una serie de transformaciones sobre elementos sucesivos de datos de entrada. Los datos entran al sistema y luego navegan a través de componentes al mismo tiempo, antes de ser asignados a su destino final.

- Ventajas:
 - Permite entender el sistema global en términos de la combinación de componentes.
 - Soporta fácilmente la reutilización.
 - Los filtros son independientes de sus vecinos.
 - Facilidad de mantenimiento y evolución.
 - Facilidad de diagnóstico.
 - Soportan la ejecución concurrente.
- Desventajas:
 - No es aconsejable en sistemas que requieren interactividad.
 - Presenta problemas de rendimiento, ya que los datos se transmiten de forma completa entre filtros.

PROCESO SECUENCIAL POR LOTES (BATCH)

Es un modelo secuencial de procesamiento de datos que se caracteriza porque sus subsistemas solo pueden iniciar su ejecución una vez que la ejecución del subsistema que le precede ha completado su ejecución. El flujo de datos es completo y/o incremental filtro a filtro. Este tipo de arquitectura es común en el ámbito empresarial, como por ejemplo los sistemas de facturación y de transacciones bancarias.

- Ventajas: Normalmente, la Arquitectura BS proporciona divisiones más simples en los subsistemas. Cada subsistema puede ser un programa independiente que trabaja sobre datos de entrada y produce datos de salida.
- Desventajas: No proporciona concurrencia e interfaz interactiva, sino que proporciona alta latencia y bajo rendimiento. Además, se requiere control externo para la implementación.

ARQUITECTURAS DE LLAMADA Y RETORNO

Basadas en la descomposición jerárquica en subrutinas (componentes) que solucionan una tarea o función definida. Los datos son pasados como parámetros y el manejador principal proporciona un ciclo de control sobre las subrutinas. Reflejan la estructura del lenguaje de programación. Permite al diseñador del software construir una estructura de programa relativamente fácil de modificar y ajustar a escala. Se basan en la bien conocida abstracción de procedimientos/funciones/métodos.

ARQUITECTURA ORIENTADA A OBJETOS

Es un tipo de arquitectura de software donde los componentes, denominados objetos, encapsulan los datos y las operaciones que realizan para la manipulación de dichos datos. Los objetos interactúan a través de la invocación de funciones y procedimientos, los cuales actúan como conectores.

Los componentes de este estilo arquitectónico se basan en los principios y características de la programación orientada a objetos (POO):

- **Propiedades de la OO:**
 - **Identidad:** Los datos se cuantifican en entidades discretas y distintas denominadas objetos. Dos objetos con los mismos valores de sus atributos son dos objetos diferentes.
 - **Clasificación:** Los objetos que tienen la misma estructura de datos (atributos) y el mismo comportamiento (operaciones), están agrupados en la misma clase.
 - **Herencia:** Mecanismo mediante el cual una clase (subclase) adquiere las propiedades de otra clase jerárquicamente superior (superclase, clase base). La herencia nos permite definir una clase como extensión de otra. Existen diferentes tipos:
 - **De implementación:** la clase hija hereda la implementación de métodos de la clase padre.
 - **De interfaz:** la clase hija hereda el interfaz (pero no la implementación de las operaciones).
 - **De clase:** combina las anteriores.

También se puede clasificar la herencia en simple y múltiple (herencia de varias clases de forma simultánea). Ésta última, no soportada por todos los lenguajes OO, puede implicar ambigüedades que se resuelven a través de las ligaduras estáticas (en tiempo de compilación) o dinámicas (en tiempo de ejecución)

- **Polimorfismo:** Propiedad, consecuencia de la herencia, por la cual un mismo mensaje puede originar conductas diferentes al ser recibido por objetos diferentes. Si se produce por variación de los parámetros que definen un método (en tiempo de compilación) se denomina sobrecarga.
- **Encapsulación:** Cualidad por la que los datos de un objeto sólo pueden ser manipulados mediante los mensajes y métodos predefinidos, y que facilita la abstracción de datos.
- **Persistencia:** Cualidad que se refiere a la permanencia del objeto, es decir, al tiempo durante el cual se le asigna espacio y puede ser accesible.
- **Extensibilidad:** Capacidad de un programa para ser fácilmente alterado de forma que pueda tratar con nuevas clases de entrada. En POO se basa en los mecanismos de herencia, polimorfismo, composición y agregación.
- **Reusabilidad:** Capacidad de producir componentes reutilizables para otros diseños o aplicaciones. En OO se consigue de una forma natural mediante el diseño de los componentes para residir en algún repositorio estándar.

- Composición/Agregación: Representa el principio de la partición. Una clase B es parte de una clase A. La agregación proporciona relaciones del tipo “parte-de” o “formado-por”.

ARQUITECTURA DE INVOCACIÓN IMPLÍCITA BASADA EN EVENTOS

En lugar de invocaciones de procedimientos explícitas directas, un componente anuncia uno o más eventos y otros componentes registran el interés en un evento asociando un procedimiento a dicho evento. El componente que anuncia dicho evento no sabe a qué componentes va a afectar, además tales componentes pueden pertenecer a otro módulo. Esta arquitectura está presente dos tipos de modelos:

- Modelos de emisión (Broadcast): En este tipo de modelos se emiten eventos a todos los subsistemas. Estos subsistemas se “suscriben” al suprasistema que luego les delega el control de dichos eventos. Este modelo es fácilmente escalable puesto que se pueden integrar nuevos subsistemas con anonimato de los mismos. Sin embargo, estos últimos no saben con certeza cuándo se producirán estos eventos.
- Modelos dirigidos por interrupciones: Son modelos de control en los cuales la respuesta inmediata a los eventos producidos provenientes del exterior es esencial. Estos eventos son denominados interrupciones los cuales tienen un tiempo de respuesta de acuerdo a la prioridad que se le asigne. Las diferentes interrupciones son manejadas por un controlador que ejecuta un determinado proceso. Aunque su gran ventaja es la respuesta rápida a interrupciones, la validación de las ocurrencias posee alta dificultad, lo que se suma al hecho de que el hardware puede ser una limitación en lo que respecta a número de interrupciones, lo cual reduce el rendimiento.

ARQUITECTURA POR CAPAS

Es un modelo organizado jerárquicamente donde cada capa provee servicios a la capa superior y es servida por la capa inferior (modelo estricto). Los componentes son las capas y los conectores son los protocolos de interacción entre las mismas. Este modelo permite descomponer el sistema en varios niveles abstrayéndolo.

ARQUITECTURAS ORIENTADAS A LA INTERACCIÓN

Estos sistemas tienen por objetivo la separación de los módulos que engloban la estructura de funcionamiento general del software.

MODELO-VISTA-CONTROLADOR (MVC)

Este tipo de arquitectura se destaca por ser efectiva para aplicaciones que utilizan muchas ventanas sincronizadas con un mismo modelo de datos, de igual manera, estas se pueden separar/incorporar fácilmente al sistema. Por otro lado, la fuerte dependencia de dichas ventanas/controladores con el modelo limita la posibilidad de modificar estos últimos por los costos que conllevaría.

- Modelo: En este se maneja la persistencia de datos y la lógica del negocio.
- Controlador: Es el que supervisa todas las acciones del sistema y realiza la conexión entre el modelo y la vista
- Vista: Es el responsable de la presentación audiovisual de los datos de salida, así como de representar una interface para la entrada de datos.

PRESENTATION - ABSTRACTION – CONTROL (PAC)

Presenta similitudes con MVC en lo que respecta a funciones, sin embargo, la comunicación es diferente. En PAC no hay comunicación directa de la presentación con la abstracción, sino que ambos convergen al control. En sistemas de múltiples agentes, se sigue una jerarquía específica. Los agentes de alto nivel se encargan de la administración de los datos y la lógica del negocio, los de nivel medio coordinan los agentes de bajo nivel, y estos últimos se encargan de definir detalladamente la presentación de los datos.

Este tipo de arquitectura es útil para sistemas distribuidos, cuyos componentes están distantes y poseen una función específica dentro del sistema.

ARQUITECTURAS ORIENTADAS A SISTEMAS DISTRIBUIDOS

En este tipo de arquitecturas, el sistema distribuye sus componentes que están sobre una plataforma determinada. Estos tienen la capacidad de comunicarse y trabajar con otros componentes en aras de lograr un objetivo específico. De esta arquitectura son destacables características como: aprovechamiento de recursos (hardware/software), concurrencia, escalabilidad y tolerancia a fallos. Sin embargo, este tipo de arquitectura, a una gran escala suele tornarse compleja. La cantidad de componentes y conexiones es tal que se requiere una fuerte inversión en control y seguridad.

ARQUITECTURA CLIENTE-SERVIDOR

Este tipo de arquitectura es la más común en sistemas distribuidos. Descompone el sistema en dos grandes procesos: cliente, primer proceso que hace una petición al segundo proceso, y servidor: segundo proceso que recibe la petición y devuelve una respuesta al cliente.

La aplicación es vista como una serie de servicios que ofrece un servidor que son consumidos por una serie de clientes. Los servidores no necesitan saber mucho acerca de los clientes, pero estos últimos deben conocer la identidad del servidor para establecer una conexión.

En esta arquitectura pueden darse dos casos particulares basados en la funcionalidad del cliente:

- Cliente ligero (Thin Client): Este tipo de cliente solo cumple con funciones de ejecución del software de presentación, mientras que el servidor se encarga de la gestión y procesamiento de la información. Se destaca por ser útil en aplicaciones de red pequeñas donde el tráfico de información es relativamente pequeño por lo que la misma aplicación puede actuar como servidor.
- Cliente pesado (Thick/Fat Client): Este tipo de cliente tiene más responsabilidades que el cliente ligero. Mientras que el servidor solo es responsable de la administración de los datos, el software del cliente implementa toda la lógica y la presentación de información al usuario. Si bien este tipo de arquitectura distribuye mejor el procesamiento, implica mayor complejidad para el mantenimiento de los clientes pesados del sistema.

ARQUITECTURA MULTICAPA (MULTI-TIER ARCHITECTURE)

Esta arquitectura es similar a la de cliente-servidor en lo que respecta a los roles que existen en la misma, sin embargo, se diferencia en que cada una de las funciones (presentación, control y administración de datos) están separados. Esto permite que el acoplamiento de nuevas capas o

subcapas se realice de forma sencilla, así como la oportunidad de crear plantillas útiles para otras aplicaciones.

Esta arquitectura se caracteriza por tener mejor rendimiento y mayor simplicidad que la del *thin client*, siendo fácilmente escalable en la medida que se agreguen más servidores, y resultando más fácil de modificar la capa de procesamiento de la aplicación, dada su centralidad.

PEER-TO-PEER (P2P)

Es un tipo de arquitectura que permite la comunicación directa de clientes o pares (*peers*) sin necesidad de un servidor. Es la arquitectura usada por Skype para ofrecer servicios de VoIP, por el navegador Tor para dar anonimato a sus clientes, por sistemas de mensajería instantánea, o por sistemas de intercambio de archivos.

Su gran ventaja es que permite sacar provecho de los recursos de cada nodo (cliente) para ofrecer servicios a otros nodos, por lo que los servidores no sobresaturan, cosa que suele pasar en sistemas centralizados. Esta arquitectura tiene dos variantes:

- Arquitectura descentralizada: El enlace de los nodos es arbitrario y no hay una jerarquización en la red que estos componen. Las peticiones en este tipo de arquitecturas suelen tardar mucho ya que tienen que recorrer nodo a nodo toda la red para encontrar aquel/aquellos que puedan ofrecer los servicios requeridos.
- Arquitectura semicentralizada: La red global se divide en localidades y se hace uso de un DHT (*Distributed Hash Table*) que asigna determinadas responsabilidades a determinados nodos. De esa forma, cuando un usuario realiza una petición, ésta se puede redirigir para hacerla más rápida y efectiva.

Esta arquitectura es fuertemente criticada por ser la utilizada por sistemas de transferencia de archivos que violan los derechos de propiedad. Adicionalmente, es muy susceptible a la entrada de malware y robo de información.

ARQUITECTURA ORIENTADA A SERVICIOS (SOA)

Esta arquitectura tiene como base un diseño de tipo cliente/servidor con enfoque empresarial que busca ofrecer servicios de software a consumidores de servicios de software por medio de una aplicación. Estos servicios son independientes y accesibles a través de una interfaz de servicio web por medio de un protocolo específico. La SOA tiene los siguientes componentes:

- Proveedor de servicios: Quienes desarrollan y ofertan servicios a los clientes.
- Solicitante de servicios: Quien consume el servicio y lo enlaza con su aplicación

ARQUITECTURAS DE SISTEMAS ESPECIALES

SISTEMA EN TIEMPO REAL

Los sistemas de tiempo real son aquellos que reciben información del entorno donde se encuentran, comprobando su estado y actuando sobre él para restituir la normalidad si se produjeran desviaciones significativas en su comportamiento o situación. Se suelen emplear, fundamentalmente, en aplicaciones dedicadas a construir sistemas de control.



Sus objetivos principales son: respuesta inmediata, información actualizada, restricciones de funcionamiento en la ejecución de procesos, inactividad en ausencia de eventos, alto rendimiento.

Se caracterizan principalmente por: manejar interrupciones, manejar prioridades, gestionar memoria real (no virtual), utilizar mecanismos de sincronización para el acceso a datos y recursos compartidos, realizar comunicación entre procesos, bloquear los procesos en espera y gestionar colas de procesos.

SISTEMAS TRANSACCIONALES

Son arquitecturas destinadas a la gestión eficiente de transacciones, donde una transacción consiste en un conjunto de eventos que deben ser llevados a cabo como una unidad indivisible de trabajo, en la que todos y cada uno de ellos tienen éxito o todos y cada uno de ellos son rechazados.

Para la implementación de sistemas transaccionales se utilizan monitores de transacciones, que engloban un conjunto de componentes que garantizan las propiedades ACID (Atomicity, Consistency, Isolation, Durability):

- Atomicidad: cualquier cambio de estado que produce una transacción es atómico, por lo que todos los eventos que la componen deben ejecutarse o rechazarse en conjunto.
- Consistencia: Todos los cambios provocados por la transacción deben dejar al sistema en un estado correcto.
- Aislamiento: propiedad que asegura que no se afectarán entre sí las transacciones sobre los mismos datos.
- Durabilidad: propiedad que asegura la persistencia de una transacción una vez que fue aceptada.

ARQUITECTURAS DE OBJETOS DISTRIBUIDAS

Dando una visión general, en una arquitectura de objetos distribuidos cada entidad distribuida es un objeto que provee y recibe servicios a y desde otros objetos. Los objetos pueden ser distribuidos entre un número diverso de computadoras en una red y comunicarse a través de una capa intermedia llamada middleware. Este middleware actúa como un bus de software que proporciona un conjunto de servicios que permiten a los objetos comunicarse, y que recibe el nombre de ORB (objeto intermediario de solicitudes).

Existen diferentes tecnologías que implementan la comunicación entre los objetos distribuidos, entre las que pueden destacarse RMI, EJB, DCOM y CORBA, así como las relacionadas con los servicios web distribuidos (SOAP, WSDL, UDDI) y los agentes móviles.

2. DIAGRAMAS DE DESPLIEGUE

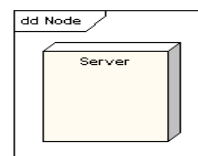
Los diagramas de despliegue muestran la configuración en funcionamiento del sistema incluyendo su software y su hardware.

Complementan a los diagramas de componentes de forma que, juntos, proveen la vista de implementación del sistema, describiendo la topología del sistema, es decir, la estructura de los elementos de hardware y el software que ejecuta cada uno de ellos, representando a los nodos y sus relaciones.

Se utilizan principalmente en sistemas empujados (colección de hardware con una gran cantidad de software que interactúa con el mundo físico), sistemas cliente-servidor y sistemas completamente distribuidos.

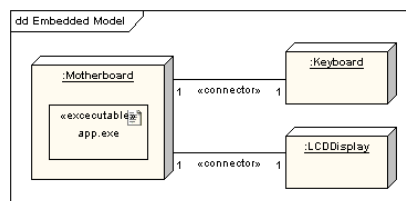
2.1. COMPONENTES Y NOTACIÓN

- **Nodo:** objeto físico en tiempo de ejecución que representa un recurso computacional, generalmente con memoria y capacidad de procesamiento. Un nodo es un elemento de hardware o software. Se representa utilizando un cubo tridimensional.

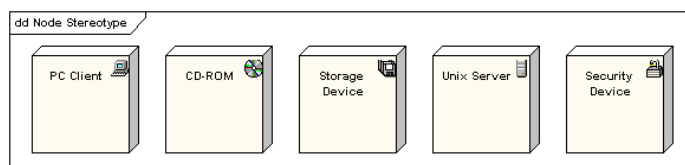
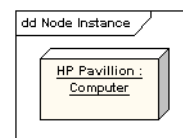


Un nodo a su vez puede tener nodos incluidos en su interior (p.e. un nodo servidor de BBDD con dos BBDD diferentes), dando a conocer que son sistemas separados incluidos dentro del mismo nodo físico (nodos compuestos).

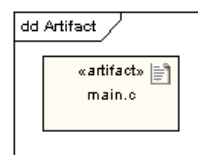
Igualmente, un nodo puede contener otros elementos, como componentes o artefactos. El siguiente diagrama muestra una parte de un sistema embebido que integra un artefacto ejecutable contenido por el nodo madre (*motherboard*).



- **Instancia de nodo:** Una instancia se puede distinguir desde un nodo por el hecho de que su nombre está subrayado y tiene dos puntos antes del tipo de nodo base. Una instancia puede o no tener un nombre antes de los dos puntos.
- **Estereotipo de nodo:** son elementos u objetos que se repiten sin variación. El estereotipo de un nodo es la forma de verificar el tipo de nodo que se está observando.



- **Artefactos:** Un artefacto es un producto del proceso de desarrollo de software, que puede incluir los modelos del proceso (modelos de Caso de uso, modelos de Diseño, etc.), archivos fuente, ejecutables, documentos de diseño, reportes de prueba, prototipos, manuales de usuario, etc., representando por tanto un artefacto un conjunto de componentes.
- **Asociación:** representan una conexión entre dos nodos, a través de la cual éstos intercambian información en forma de mensajes o señales. Se representa con una línea continua que une los dos nodos.



Es común incluir en las conexiones una etiqueta que represente a través de que medio se realiza la conexión (Internet, WAN...). También, si es relevante, se suele poner al lado de los nodos el número de nodos que participan en la asociación.

